

Учебная методичка: как сверстать страницу «Научиться учиться» с нуля

Мы будем дописывать только `index.html` и `style.css` из шаблона, который я вам дал

Цель урока

Бриф задаёт шесть крупных блоков, сетку, размеры, цвета, типографику, отступы и правила именования. В готовом HTML эта структура реализована семантическими тегами.

1. Начинаем не с CSS, а с брифа

Профессиональная вёрстка начинается с анализа задачи.

Перед кодом выписываем:

- крупные секции;
- повторяющиеся компоненты;
- размеры;
- цвета;
- типографику;
- отступы;
- декоративные элементы;
- требования к семантике;
- правила именования классов.

В нашем брифе страница состоит из `header`, `description`, `digits`, `feynman`, `kaufman`, `footer`; внутри страницы используется общий контейнер `page`.

Главный принцип:

Сначала понимаем структуру, потом выбираем CSS-инструменты.

2. Создаём HTML-каркас

Начинаем с минимального документа:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Научиться учиться</title>
  <link rel="stylesheet" href="./styles/normalize.css">
  <link rel="stylesheet" href="./styles/style.css">
</head>
<body>
  <div class="page">
    <header class="header"></header>
    <main class="content"></main>
    <footer class="footer"></footer>
  </div>
</body>
</html>
```

Готовый проект именно так подключает `normalize.css` и `style.css`, а содержимое помещает внутрь `.page`.

Почему семантические теги?

`header`, `main`, `section`, `footer`, `nav` описывают **смысл документа**, а не его внешний вид. Бриф прямо требует размечать общую структуру семантическими тегами.

Поэтому не начинаем с бесконечных `<div>`.

3. `.page` : первый CSS

```
.page {
  min-width: 1100px;
  max-width: 1600px;
  margin: 0 auto;
  font-family: 'Helvetica Neue', 'Arial', sans-serif;
}
```

`min-width: 1100px`

Страница не становится уже 1100px. В этом учебном проекте мобильная версия ещё не реализуется.

```
max-width: 1600px
```

На очень широком мониторе содержимое не растягивается бесконечно.

```
margin: 0 auto
```

Если ширина окна больше максимальной ширины страницы, свободное пространство делится между левым и правым `margin`, поэтому `.page` оказывается по центру.

Цепочка шрифтов

```
font-family: 'Helvetica Neue', 'Arial', sans-serif;
```

Сначала браузер пытается использовать Helvetica Neue, затем Arial, затем системный шрифт без засечек. Это непосредственно задано брифом.

4. Почему нельзя доверять браузерным отступам

У заголовков, абзацев и списков браузер имеет собственные `margin` и `padding`. `normalize.css` не отменяет все из них.

Поэтому если дизайн говорит, что между двумя элементами должно быть 30px, мы задаём это сами:

```
.two-columns__paragraph {  
  margin-top: 0;  
  margin-bottom: 30px;  
}
```

Бриф специально требует не оставлять расстояния, унаследованные от браузера.

5. БЭМ: почему классы называются именно так

В шапке мы получаем:

```
header  
header__title
```

```
header__subtitle
header__link
header__main-illustration
header__square-pic
```

`header` — блок.

`header__title` — элемент блока.

Такое именование позволяет видеть структуру прямо по названию класса. Бриф использует именно эту модель и отдельно требует классы элементов шапки.

6. Почему у логотипа два класса

В HTML:

```
<img class="logo logo_place_header" ...>
```

Первый класс описывает компонент:

```
logo
```

Второй — его конкретное расположение:

```
logo_place_header
```

В подвале тот же компонент можно использовать повторно:

```
<img class="logo logo_place_footer" ...>
```

Это важная инженерная идея: **не создавать новый компонент только потому, что объект находится в другом месте страницы**. Бриф прямо предлагает переиспользовать `logo`.

7. Шапка: сначала HTML

```

<header class="header">
  

  <h1 class="header__title">Научиться учиться</h1>

  <p class="header__subtitle">
    Какие современные и эффективные подходы к обучению вы можете
    использовать в своей жизни?
    <a href="#" class="header__link">Узнать →</a>
  </p>

  <div class="header__square-pic"></div>
</header>

```

Готовый HTML содержит именно эту композицию: логотип, `h1`, подзаголовок со ссылкой, основную иллюстрацию и декоративный квадрат.

8. Почему `.header` — `position: relative`

```

.header {
  position: relative;
}

```

Само по себе это почти незаметно. Но внутри есть элементы с `position: absolute`.

`position: relative` делает `.header` системой координат для таких дочерних элементов.

Можно мыслить так:

```

header = координатная система
|

```

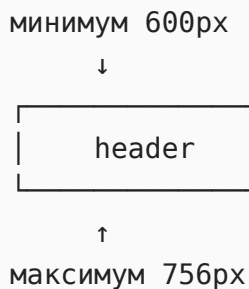
└─ logo	→ absolute
└─ illustration	→ absolute
└─ square	→ absolute

9. Почему высота шапки задаётся тремя свойствами

```
.header {  
  height: 100vh;  
  min-height: 600px;  
  max-height: 756px;  
}
```

100vh означает высоту, равную высоте области просмотра браузера.

Но одной 100vh недостаточно. Поэтому задаём ограничения:



Бриф объясняет это тем, что большая часть элементов шапки имеет абсолютное позиционирование и поэтому находится вне обычного потока.

10. Где здесь Flexbox

```
.header {  
  display: flex;  
  align-items: center;  
}
```

Flexbox нужен для вертикального выравнивания основного заголовка.

При этом:

```
margin-left: 64px;
```

отвечает за его горизонтальное смещение.

Это хороший пример принципа:

Один инструмент может решать одну задачу, а другое свойство — другую.

Брифф именно так описывает подход к заголовку шапки.

11. Логотип

```
.logo_place_header {  
  position: absolute;  
  top: 30px;  
  left: 64px;  
}
```

Логотип должен находиться в конкретной точке шапки, независимо от заголовка. Поэтому он выводится из обычного потока через `absolute`.

Важно, что позиционирование задаётся модификатором `logo_place_header`, а не базовым `logo`.

12. Главный заголовок

```
.header__title {  
  width: 730px;  
  margin-left: 64px;  
  line-height: 96px;  
  font-size: 102px;  
  font-weight: bold;  
}
```

Читаем не по отдельности, а как систему:

- `width` ограничивает область текста;
- `margin-left` задаёт положение относительно левого края;

- `font-size` определяет размер букв;
- `line-height` определяет высоту строки;
- `font-weight` задаёт начертание.

Бриф задаёт для заголовка размер 102px и высоту строки 96px.

13. Подзаголовок и `bottom`

```
.header__subtitle {  
  position: absolute;  
  width: 388px;  
  left: 64px;  
  bottom: 30px;  
  z-index: 3;  
}
```

Почему `bottom`, а не `margin-top`?

Потому что нам нужна привязка к **нижней границе шапки**, а не к предыдущему элементу.

Это фундаментальная разница между позиционированием в потоке и абсолютным позиционированием.

14. Слои и `z-index`

В шапке объекты могут перекрываться:

```
.header__square-pic {  
  z-index: 0;  
}  
  
.header__main-illustration {  
  z-index: 1;  
}  
  
.header__subtitle,  
.logo_place_header {  
  z-index: 3;  
}
```

Условная модель:

```
3  текст и логотип
2  —
1  изображение
0  квадрат
```

`z-index` управляет порядком наложения позиционированных элементов.

15. Главная иллюстрация

```
.header__main-illustration {
  position: absolute;
  bottom: 0;
  right: 0;
  width: 765px;
  height: 608px;
}
```

Мы читаем это как:

Прикрепить изображение к правому нижнему углу шапки и задать ему размеры из макета.

В HTML обязательно оставляем `alt`. Бриф отдельно напоминает об этом требовании.

16. Декоративный квадрат

```
.header__square-pic {
  position: absolute;
  right: 0;
  top: 64px;
  width: 568px;
  height: 568px;
  background-color: #2F80ED;
  z-index: 0;
}
```

Это декоративный объект, поэтому ему не нужно участвовать в обычном потоке документа.

Цвет берётся из брифа, а не выбирается «примерно похожий». Для синего задан #2F80ED .

17. Секция `description` : строим две колонки

```
<section class="description">
  <div class="two-columns">
    <h2 class="two-columns__brief">Главные проблемы в обучении</h2>

    <div class="two-columns__main-text">
      <p class="two-columns__paragraph">...</p>
      <p class="two-columns__paragraph">...</p>
      <p class="two-columns__paragraph">...</p>
    </div>
  </div>
</section>
```

Задача: получить две колонки на одной строке.

Для этого естественный инструмент — Flexbox.

18. Центрирование контейнера

```
.description {
  display: flex;
  justify-content: center;
}
```

`.description` становится Flex-контейнером, а `.two-columns` оказывается по центру.

Далее уже сам `.two-columns` управляет двумя внутренними колонками.

Бриф прямо рекомендует Flexbox для горизонтального центрирования.

19. space-between

```
.two-columns {  
  display: flex;  
  justify-content: space-between;  
  width: 80%;  
  margin-top: 100px;  
  margin-bottom: 100px;  
}
```

space-between означает:

первый элемент прижать к началу, последний — к концу, а свободное место распределить между ними.

Получается:

brief



main-text



20. width и min-width

```
.two-columns__main-text {  
  width: 80%;  
  min-width: 784px;  
}
```

width задаёт обычную ширину, а min-width не позволяет блоку стать меньше заданного порога.

Это пример сочетания относительного размера и ограничения.

21. Почему span для «Вывод:»

```
<p class="two-columns__paragraph">
  <span class="two-columns__span-accent">Вывод:</span>
  учиться тоже нужно уметь...
</p>
```

Вывод: — не отдельный абзац. Это фрагмент того же абзаца.

Поэтому используем строчный контейнер `span`:

```
.two-columns__span-accent {
  font-weight: bold;
}
```

Бриф прямо указывает использовать `span` для такого выделения.

22. Универсальные компоненты

В секции `digits` появляются:

```
section-title
section-subtitle
table
table__cell
table__heading
table__text
```

Это принципиальное архитектурное решение.

Мы не пишем:

```
digits__title
digits__table
```

потому что эти компоненты понадобятся повторно.

Бриф прямо требует проектировать эти блоки так, чтобы использовать их в других секциях с модификаторами.

23. Универсальный заголовок секции

```
.section-title {  
  display: flex;  
  justify-content: center;  
  margin-top: 100px;  
  margin-bottom: 20px;  
  width: 60%;  
  line-height: 69px;  
  font-size: 60px;  
  font-weight: bold;  
}
```

Это **компонент**, а не стиль конкретной секции.

Поэтому его можно использовать и в другой части страницы.

24. «Таблица» через Flexbox

HTML:

```
<div class="table">  
  <div class="table__cell">  
    <h3 class="table__heading">86 миллиардов</h3>  
    <p class="table__text">Число нейронов в мозге человека</p>  
  </div>  
  ...  
</div>
```

CSS:

```
.table {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  flex-wrap: wrap;  
  width: 1100px;  
}
```

`flex-wrap: wrap` разрешает перенос ячеек на следующую строку.

Важно: это визуальная сетка карточек, а не таблица табличных данных. Поэтому `<div>` здесь логичнее настоящего `<table>`.

Бриф прямо предлагает Flexbox для автоматического распределения ячеек.

25. Почему `h3`, а не `div`

```
<h3 class="table__heading">86 миллиардов</h3>
```

Это заголовок отдельного пункта, поэтому используем семантический `h3`.

Размер шрифта — это CSS-свойство. Семантический уровень заголовка — свойство HTML.

Не надо путать эти две вещи.

Бриф отдельно требует следить за семантикой заголовков.

26. Метод Фейнмана

```
<section class="feinman">
  <h2 class="feinman__title">Метод Фейнмана</h2>
  <p class="feinman__subtitle">Выучить и не забыть.</p>
  <a href="#" class="feinman__link">Подробнее →</a>
  
</section>
```

Сама секция:

```
.feinman {
  display: flex;
  flex-direction: column;
  position: relative;
  min-height: 790px;
  background-color: #F2F2F2;
}
```

Почему снова `position: relative`?

Потому что картинка и ссылка будут абсолютно позиционироваться относительно этой секции.

Бриф допускает использование фоновой картинки, но также предлагает `<img>` с абсолютным позиционированием.

27. Картинка Фейнмана

```
.feinman__pic {  
  width: 867px;  
  height: 637px;  
  position: absolute;  
  left: 0;  
  bottom: 0;  
}
```

Читаем:

Картинка не должна влиять на поток текста. Она прикреплена к левому нижнему углу секции.

Это хороший пример ситуации, когда `absolute` действительно оправдан.

28. Ссылка «Подробнее»

```
.feinman__link {  
  position: absolute;  
  top: 50%;  
  right: 48px;  
}
```

Нам нужна привязка к правой стороне секции и примерно к её середине.

Поэтому здесь удобнее абсолютное позиционирование. Бриф прямо указывает расположить ссылку примерно на высоте половины секции.

29. Самый важный момент: повторное использование `table`

В `kaufman` HTML выглядит так:

```
<section class="kaufman">
  <h2 class="section-title section-title_theme_dark">
    Принципы обучения
  </h2>

  <p class="section-subtitle section-subtitle_theme_dark">
    от Джоша Кауфмана
  </p>

  <div class="table table_theme_dark">
    ...
  </div>
</section>
```

Мы повторно используем:

```
table
table__cell
table__heading
table__text
```

и добавляем модификаторы:

```
table_theme_dark
table__cell_theme_dark
table__heading_theme_dark
table__text_theme_dark
```

Бриф именно это и требует: повторно использовать компонент `table`, а уникальные стили описывать модификаторами.

30. Что такое модификатор

Базовый компонент:

```
.table__text {  
  color: #000000;  
}
```

В тёмной теме:

```
.table__text_theme_dark {  
  color: white;  
}
```

HTML:

```
<p class="table__text table__text_theme_dark">
```

Первый класс говорит **что это**.

Второй — **в каком варианте это находится**.

Это гораздо лучше, чем создавать отдельный набор почти одинаковых компонентов.

31. Почему нельзя дублировать CSS

Плохая архитектура:

```
.digits__cell { ... }  
.kaufman__cell { ... }
```

если структура обеих ячеек одинаковая.

Лучше:

```
.table__cell { ... }  
.table__cell_theme_dark { ... }
```

Получаем:

```
базовый компонент  
  +  
модификатор
```



новый вариант

Так CSS становится меньше, понятнее и переиспользуемое.

32. Декоративный треугольник

```
.kaufman__triangle {  
  position: absolute;  
  width: 877px;  
  height: 877px;  
  right: -210px;  
  top: 0;  
  z-index: -1;  
}
```

`right: -210px` означает, что объект частично выходит за правую границу родительского блока.

Бриф специально требует такое смещение.

33. Зачем `overflow: hidden`

```
.kaufman {  
  overflow: hidden;  
}
```

Треугольник выходит за пределы секции, но мы не хотим видеть эту часть.

`overflow: hidden` обрезает содержимое, выходящее за границы блока.

Таким образом:

большой декоративный объект



выходит за границу





Бриф непосредственно связывает отрицательное позиционирование треугольника с `overflow: hidden`.

34. Footer: снова Flexbox

```
.footer {  
  display: flex;  
  min-height: 350px;  
  justify-content: center;  
  background-color: #1F1F1F;  
}
```

Внутри:

```
.footer__columns {  
  display: flex;  
  justify-content: space-between;  
  width: 90%;  
  margin-top: 60px;  
  margin-bottom: 40px;  
}
```

Получаем три смысловые колонки: copyright, информация и социальные сети. В HTML для информационных и социальных колонок используется `nav`.

35. Почему ссылки находятся в `ul`

```
<ul class="footer__column-links">  
  <li><a href="#">Главная</a></li>  
  <li><a href="#">Концепция</a></li>  
  <li><a href="#">Наставники</a></li>  
</ul>
```

Это список ссылок, поэтому HTML выражает смысл через `ul/li`, а CSS уже убирает стандартные маркеры и выстраивает элементы вертикально:

```
.footer__column-links {  
  display: flex;  
  flex-direction: column;  
  list-style: none;  
  padding-left: 0;  
}
```

36. Почему иконка находится внутри `<a>`

```
<a href="https://vk.com" class="footer__column-link">  
    
  ВКонтакте  
</a>
```

Пользователь должен иметь возможность нажать не только на текст, но и на иконку. Бриф прямо требует, чтобы каждая иконка была частью ссылки.

37. HTML-мнемоника `©`

```
<p class="footer__autor">  
  &copy; 2024 Владимир Андропов  
</p>
```

Браузер отображает `©` как ©.

Мнемоники удобны для специальных символов. Бриф отдельно приводит `©` как пример.

38. Где JavaScript?

В текущем проекте **JavaScript не используется**.

HTML подключает только:

```
normalize.css  
style.css
```

JavaScript-файл отсутствует.

Это важно не считать недостатком.

На данном этапе проект учит строить статический интерфейс:

```
HTML → структура  
CSS  → внешний вид и раскладка
```

JavaScript нужен тогда, когда появляется поведение:

- открытие меню;
- модальное окно;
- обработка формы;
- динамическое изменение DOM;
- интерактивность;
- получение и отображение данных.

Не следует использовать JS там, где задачу уже естественно решает CSS.

Например, два блока рядом — это задача Flexbox, а не JavaScript.

39. Главный принцип выбора CSS-инструмента

Элементы в ряд

```
display: flex;
```

Элементы вертикально

```
flex-direction: column;
```

Раздвинуть элементы

```
justify-content: space-between;
```

Центрировать

```
justify-content: center;
```

или в зависимости от оси:

```
align-items: center;
```

Разрешить перенос

```
flex-wrap: wrap;
```

Привязать декоративный объект

```
position: absolute;
```

Создать координатную систему

```
position: relative;
```

Управлять слоями

```
z-index: ...;
```

Обрезать выходящее содержимое

```
overflow: hidden;
```

40. Почему нельзя сделать всю страницу через absolute

Можно было бы написать:

```
.title {  
  position: absolute;  
  left: 64px;
```

```
top: 200px;  
}
```

и таким образом расставить каждый элемент.

Но тогда страница превратится в набор координат:

элемент → x → y

Вместо структуры:

```
страница  
├── секция  
│   ├── колонка  
│   └── колонка  
├── секция  
│   └── сетка  
└── секция  
    └── контент
```

В готовом проекте `absolute` используется там, где он действительно оправдан: декоративные изображения, квадрат, треугольник, ссылка и логотип.

Основная структура строится через нормальный поток и Flexbox.

41. Правило «сначала поток, потом исключения»

Хорошее правило начинающего frontend-разработчика:

Сначала попробуй решить задачу обычным потоком документа или Flexbox. Абсолютное позиционирование используй для элементов, которые действительно должны быть привязаны к конкретным координатам.

Например:

```
два блока рядом → Flexbox  
текст друг под другом → обычный поток / column  
декоративный квадрат → absolute
```

Это предотвращает появление «магических координат» по всей странице.

42. Как читать готовый CSS

Не надо читать:

```
width  
margin  
position  
font-size
```

как четыре независимые команды.

Нужно читать CSS как **систему ограничений**.

Например:

```
.feinman {  
  position: relative;  
}  
  
.feinman__pic {  
  position: absolute;  
  left: 0;  
  bottom: 0;  
}
```

Перевод на человеческий язык:

Секция является системой координат. Картинка исключается из обычного потока и прикрепляется к левому нижнему углу секции.

Другой пример:

```
.table {  
  display: flex;  
  flex-wrap: wrap;  
}  
  
.table__cell {  
  width: 250px;  
}
```

Перевод:

Есть горизонтальная сетка одинаковых по ширине ячеек, которые при необходимости переходят на следующую строку.

43. Еще раз по шагам

Шаг 1. Изучить бриф

Выписать структуру, размеры, цвета, типографику и компоненты.

Шаг 2. Создать семантический HTML (взять из шаблона)

Сначала:

```
header
main
section
footer
```

Шаг 3. Добавить контент

Заголовки, абзацы, ссылки, изображения, списки.

Шаг 4. Найти повторения

Например:

```
logo
section-title
section-subtitle
table
```

Шаг 5. Задать глобальные ограничения

```
.page {
  min-width: 1100px;
  max-width: 1600px;
  margin: 0 auto;
}
```

Шаг 6. Сверстать крупные секции

```
header
↓
description
↓
digits
↓
feinman
↓
kaufman
↓
footer
```

Шаг 7. Для каждой задачи выбрать инструмент

```
структура → HTML
раскладка → Flexbox
координатная привязка → absolute
слои → z-index
обрезка → overflow
типографика → font-*
```

Шаг 8. Добавить модификаторы

Например:

```
class="table table_theme_dark"
```

Шаг 9. Сравнить с макетом

Проверять не только «похоже ли», а конкретные параметры:

- ширина;
 - высота;
 - отступы;
 - положение;
 - цвет;
 - размер шрифта;
 - высота строки;
 - порядок слоёв.
-

44. Контрольные вопросы

1. Почему `.page` имеет одновременно `min-width` и `max-width`?
 2. Что делает `margin: 0 auto`?
 3. Почему `.header` имеет `position: relative`?
 4. Почему логотип позиционируется абсолютно?
 5. Чем `font-size` отличается от `line-height`?
 6. Что делает `z-index`?
 7. Почему основная структура не сделана целиком через `absolute`?
 8. Зачем `.description` нужен Flexbox?
 9. Что делает `space-between`?
 10. Зачем таблице `flex-wrap: wrap`?
 11. Почему визуальная таблица сделана через `div`, а не `<table>`?
 12. Зачем нужны `h2` и `h3` вместо обычных `div`?
 13. Что означает `table__text_theme_dark`?
 14. Зачем `right: -210px` у треугольника?
 15. Зачем `overflow: hidden`?
 16. Почему иконка соцсети находится внутри `<a>`?
 17. Почему в этом проекте JavaScript не нужен?
 18. Как понять, что задачу следует решать CSS, а не JavaScript?
-

Итоговая архитектура проекта (классов для css)

```
.page
|
|— header
|   |— logo
|   |— header__title
|   |— header__subtitle
|   |   |— header__link
|   |— header__main-illustration
|   |— header__square-pic
|
|— content
|   |— description
|       |— two-columns
|           |— two-columns__brief
|           |— two-columns__main-text
|           |— two-columns__paragraph
```

```
|
|
|└─ digits
|   |└─ section-title
|   |└─ section-subtitle
|   |└─ table
|       └─ table__cell
|           └─ table__heading
|           └─ table__text
|
|└─ feinman
|   |└─ feinman__title
|   |└─ feinman__subtitle
|   |└─ feinman__link
|   |└─ feinman__pic
|
|└─ kaufman
|   |└─ section-title_theme_dark
|   |└─ section-subtitle_theme_dark
|   |└─ table_theme_dark
|   |└─ kaufman__triangle
|
└─ footer
    └─ footer__columns
        └─ copyright
        └─ info
        └─ social
```
